
A Simple Weight Decay Can Improve Generalization

Anders Krogh*

CONNECT, The Niels Bohr Institute
Blegdamsvej 17
DK-2100 Copenhagen, Denmark
krogh@cse.ucsc.edu

John A. Hertz

Nordita
Blegdamsvej 17
DK-2100 Copenhagen, Denmark
hertz@nordita.dk

Abstract

It has been observed in numerical simulations that a weight decay can improve generalization in a feed-forward neural network. This paper explains why. It is proven that a weight decay has two effects in a linear network. First, it suppresses any irrelevant components of the weight vector by choosing the smallest vector that solves the learning problem. Second, if the size is chosen right, a weight decay can suppress some of the effects of static noise on the targets, which improves generalization quite a lot. It is then shown how to extend these results to networks with hidden layers and non-linear units. Finally the theory is confirmed by some numerical simulations using the data from NetTalk.

1 INTRODUCTION

Many recent studies have shown that the generalization ability of a neural network (or any other 'learning machine') depends on a balance between the information in the training examples and the complexity of the network, see for instance [1,2,3]. Bad generalization occurs if the information does not match the complexity, *e.g.* if the network is very complex and there is little information in the training set. In this last instance the network will be over-fitting the data, and the opposite situation corresponds to under-fitting.

*Present address: Computer and Information Sciences, Univ. of California Santa Cruz, Santa Cruz, CA 95064.

Often the number of free parameters, *i.e.* the number of weights and thresholds, is used as a measure of the network complexity, and algorithms have been developed, which minimizes the number of weights while still keeping the error on the training examples small [4,5,6]. This minimization of the number of free parameters is not always what is needed.

A different way to constrain a network, and thus decrease its complexity, is to limit the growth of the weights through some kind of weight decay. It should prevent the weights from growing too large unless it is really necessary. It can be realized by adding a term to the cost function that penalizes large weights,

$$E(\mathbf{w}) = E_0(\mathbf{w}) + \frac{1}{2}\lambda \sum_i w_i^2, \quad (1)$$

where E_0 is one's favorite error measure (usually the sum of squared errors), and λ is a parameter governing how strongly large weights are penalized. $\mathbf{w}$ is a vector containing all free parameters of the network, it will be called the weight vector. If gradient descend is used for learning, the last term in the cost function leads to a new term $-\lambda w_i$ in the weight update:

$$\dot{w}_i \propto -\frac{\partial E_0}{\partial w_i} - \lambda w_i. \quad (2)$$

Here it is formulated in continuous time. If the gradient of E_0 (the 'force term') were not present this equation would lead to an exponential decay of the weights.

Obviously there are infinitely many possibilities for choosing other forms of the additional term in (1), but here we will concentrate on this simple form.

It has been known for a long time that a weight decay of this form can improve generalization [7], but until now not very widely recognized. The aim of this paper is to analyze this effect both theoretically and experimentally. Weight decay as a special kind of regularization is also discussed in [8,9].

2 FEED-FORWARD NETWORKS

A feed-forward neural network implements a function of the inputs that depends on the weight vector $\mathbf{w}$, it is called f_w . For simplicity it is assumed that there is only one output unit. When the input is ξ the output is $f_w(\xi)$. Note that the input vector is a vector in the N -dimensional input space, whereas the weight vector is a vector in the weight space which has a different dimension W .

The aim of the learning is not only to learn the examples, but to learn the underlying function that produces the targets for the learning process. First, we assume that this *target function* can actually be implemented by the network. This means there exists a weight vector $\mathbf{u}$ such that the target function is equal to f_u . The network with parameters $\mathbf{u}$ is often called the *teacher*, because from input vectors it can produce the right targets. The sum of squared errors is

$$E_0(\mathbf{w}) = \frac{1}{2} \sum_{\mu=1}^p [f_u(\xi^\mu) - f_w(\xi^\mu)]^2, \quad (3)$$

$$\dot{w}_i \propto \sum_{\mu} [f_u(\xi^\mu) - f_w(\xi^\mu)] \frac{\partial f_w(\xi)}{\partial w_i} - \lambda w_i. \quad (4)$$

Now the idea is to expand this around the solution $\mathbf{u}$, but first the linear case will be analyzed in some detail.

3 THE LINEAR PERCEPTRON

The simplest kind of ‘network’ is the linear perceptron characterized by

$$f_w(\xi) = N^{-1/2} \sum_i w_i \xi_i \quad (5)$$

where the $N^{-1/2}$ is just a convenient normalization factor. Here the dimension of the weight space (W) is the same as the dimension of the input space (N).

The learning equation then takes the simple form

$$\dot{w}_i \propto \sum_{\mu} N^{-1} \sum_j [u_j - w_j] \xi_j^\mu \xi_i^\mu - \lambda w_i. \quad (6)$$

Defining

$$v_i \equiv u_i - w_i \quad (7)$$

and

$$A_{ij} = N^{-1} \sum_{\mu} \xi_i^\mu \xi_j^\mu \quad (8)$$

it becomes

$$\dot{v}_i \propto - \sum_j A_{ij} v_j + \lambda (u_i - v_i). \quad (9)$$

Transforming this equation to the basis where $\mathbf{A}$ is diagonal yields

$$\dot{v}_r \propto -(A_r + \lambda) v_r + \lambda u_r, \quad (10)$$

where A_r are the eigenvalues of $\mathbf{A}$, and a subscript r indicates transformation to this basis. The generalization error is defined as the error averaged over the distribution of input vectors

$$\begin{aligned} F &= \langle [f_u(\xi) - f_w(\xi)]^2 \rangle_{\xi} = \langle N^{-1} (\sum_i v_i \xi_i)^2 \rangle_{\xi} = N^{-1} \sum_{ij} v_i v_j \langle \xi_i \xi_j \rangle_{\xi} \\ &= N^{-1} \sum_i v_i^2. \end{aligned} \quad (11)$$

Here it is assumed that $\langle \xi_i \xi_j \rangle_{\xi} = \delta_{ij}$. The generalization error F is thus proportional to $|\mathbf{v}|^2$, which is also quite natural.

The eigenvalues of the covariance matrix $\mathbf{A}$ are non-negative, and its rank can easily be shown to be less than or equal to p . It is also easily seen that all eigenvectors belonging to eigenvalues larger than 0 lies in the subspace of weight space spanned